bcast/chain

An AgentMPI collective scaling analysis

Abstract

The chain broadcast passes the artifact along a line: rank i receives from $i-1$ and forwards to $i+1$. It sends $p-1$ messages, exactly as `flat` and `binomial` do, and it pays $p-1$ hops for them, so it is the algorithm that isolates depth as a variable while holding traffic fixed. Across 21 measured sizes from $p=2$ to $p=32$ every size logged $p-1$ messages, reported $p-1$, and recorded $p-1$ rounds: both panels of Figure 1 are the same straight line, and there is nothing for a non-power-of-two size to do differently because the algorithm never branches on p. What the sweep buys is the price of depth, measured: the maximum per-rank blocking time grows from 10.4 ms at $p=2$ to 1,481.8 ms at $p=32$, thirty-three times the binomial tree's 44.8 ms at the same size, against a predicted ratio of 6.2. The excess is the per-hop cost itself rising with p, from 10.4 to 47.8 ms, as thirty-two idle ranks poll one SQLite file. The single most important thing to take away is that this is the whole bill: across all thirteen sizes the chain's $p-1$ messages carried exactly *one* distinct content digest, and `fold_depth` was 0 in every record, so thirty-one hops of forwarding delivered byte-identical content to rank 31. Chain broadcast is slow and it is faithful, and the mirrored `reduce/chain` is slow and is not — that asymmetry is the point of having both.

1 What this family is

The `bcast` collective under the `chain` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.012–1.499 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

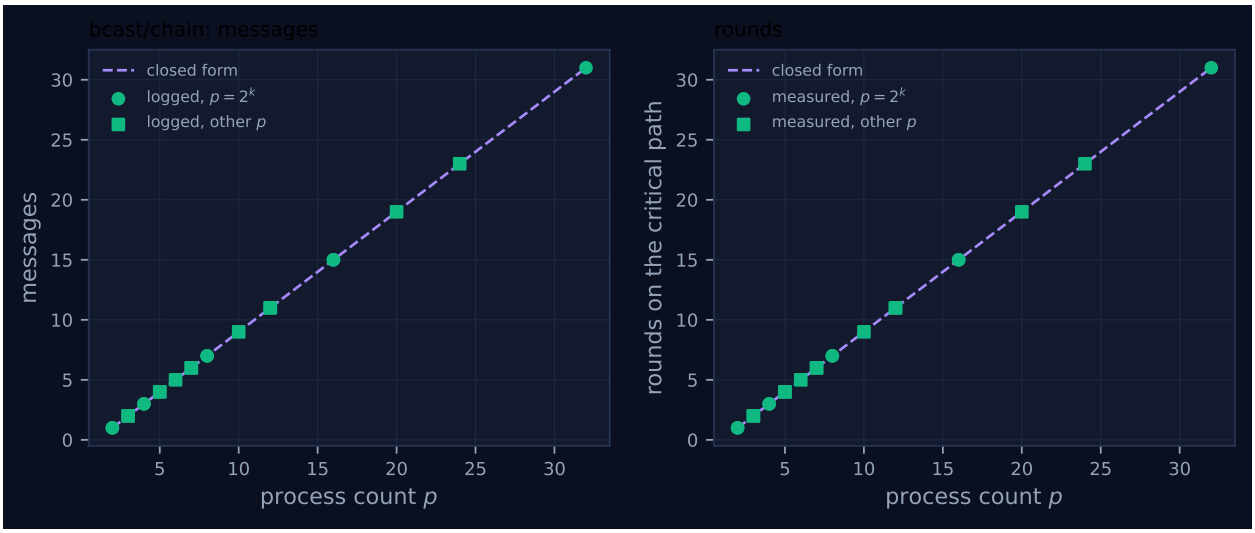

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.012	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.012	✓	✓
3	`mb-free`	2	2	2	2	2	0.025	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.016	✓	✓
4	`mb-free`	3	3	3	3	3	0.027	✓	✓
4	`mb-smoke`	3	3	3	3	3	0.028	✓	✓
5	`mb-free`	4	4	4	4	4	0.029	✓	✓
5	`mb-smoke`	4	4	4	4	4	0.029	✓	✓
6	`mb-free`	5	5	5	5	5	0.052	✓	✓
7	`mb-free`	6	6	6	6	6	0.049	✓	✓
7	`mb-smoke`	6	6	6	6	6	0.056	✓	✓
8	`mb-free`	7	7	7	7	7	0.068	✓	✓
8	`mb-smoke`	7	7	7	7	7	0.097	✓	✓
10	`mb-free`	9	9	9	9	9	0.135	✓	✓
12	`mb-free`	11	11	11	11	11	0.101	✓	✓
12	`mb-smoke`	11	11	11	11	11	0.057	✓	✓
16	`mb-free`	15	15	15	15	15	0.185	✓	✓
16	`mb-smoke`	15	15	15	15	15	0.141	✓	✓
20	`mb-free`	19	19	19	19	19	0.586	✓	✓
24	`mb-free`	23	23	23	23	23	0.760	✓	✓
32	`mb-free`	31	31	31	31	31	1.499	✓	✓

4 How the algorithm scales

The implementation is a pipeline with one wrap-around guard. The root sends to $(r+1) \bmod p$ unless that is the root itself; every other rank receives from $(r-1) \bmod p$ and then sends onward to $(r+1) \bmod p$, again unless that is the root. So each of the $p-1$ non-root ranks receives exactly

once and each of the $p-1$ ranks other than the last sends exactly once: $p-1$ messages, $p-1$ hops, and a per-rank fan-out of one. This is exactly `FORMULAS[("bcast","chain")]`, which records $(p-1,\ p-1,\ (p-1)n,\ 0)$, and the last term — fold depth zero, since a broadcast applies no operator — holds at all 21 sizes.

The critical path here is a genuine chain of dependencies, not a bookkeeping convention, and the trace shows every link. In `mb-free__coll-bcast-chain-8` the events alternate strictly: rank 0 sends at 4.09 ms; rank 1 receives at 13.52 and sends at 13.77; rank 2 receives at 16.07 and sends at 16.27; rank 3 at 27.20 and 27.45; rank 4 at 30.87 and 31.12; rank 5 at 46.81 and 47.07; rank 6 at 52.24 and 52.50; rank 7 receives at 67.78 and records the collective at 67.89. No two hops overlap, and the interval between a reception and the corresponding forward is consistently 200–260 μs while the interval between a forward and the next reception ranges from 2.2 to 15.7 ms. The forwarding is cheap; the waiting is not, and the waiting is what $p-1$ hops buys.

The consequence is that this is the one family in the broadcast set whose wall-clock measurements are worth taking seriously. The maximum per-rank blocking time runs 10.4 ms at $p=2$, 23.6 at $p=3$, 63.8 at $p=8$, 177.2 at $p=16$, 571.1 at $p=20$, 746.6 at $p=24$ and 1,481.8 at $p=32$ — three orders of magnitude of dynamic range, against 10.4 to 28.0 ms for `flat` and 10.5 to 44.8 ms for `binomial`. But it is not linear in $p-1$, and the deviation is informative. Dividing by the hop count gives a per-hop cost of 10.4 ms at $p=2$, 9.1 at $p=8$, 11.8 at $p=16$, 30.1 at $p=20$, 32.5 at $p=24$ and 47.8 at $p=32$: roughly flat up to $p=16$ and then a factor of four above it. Nothing in the algorithm changes at $p=20$. What changes is that twenty or more ranks are simultaneously blocked in `_crecv`, polling the same SQLite fabric, while exactly one hop of useful work happens at a time. The superlinearity is contention from idle ranks, and it is a property of this harness rather than of chain broadcast.

Figure 1 is therefore the least eventful figure in the family set and says so deliberately: two identical straight lines through every marker, powers of two and other sizes alike, with no staircase and no crosses. For `chain` the message count and the round count are the same integer, so the figure's two panels are two drawings of $p-1$. Its value is as a control: the binomial panel's staircase means something only because this panel exists to show what no depth reduction looks like.

The viewer screenshot for `mb-free__coll-bcast-chain-32` shows the shape unmistakably. The stats row reports a 1.5 s span, 31 messages, 0 agent calls and $0.000. The timeline is a descending staircase: rank 0's pair of glyphs sits at the far left, rank 1's a little later, and by rank 21 the glyphs have marched to roughly 0.4 s, continuing off the visible portion of the panel toward rank 31 at 1.5 s. Every lane contains exactly two instantaneous ticks and no bars, and at any moment only one lane is doing anything. This is worth comparing with `mb-free__coll-reduce-chain-32`, whose staircase runs the other way — rank 31 first, rank 0 last, because a reduction folds from the far end toward the root. The two pictures are mirror images, which is the duality made literal.

5 Powers of two and everything else

There is no remainder path. The only arithmetic in the algorithm is $(r \pm 1) \bmod p$ and the only branch is `if nxt != root`, so the code executed at $p=7$ is the code executed at $p=8$ with one fewer iteration of the same pipeline. This is the strongest form the answer to the powers-of-two question can take for a collective: not "the remainder path behaves" but "there is no remainder path to behave".

The measurements are consistent with that. All eight non-power-of-two sizes logged $p-1$ messages

(2 at $p = 3$, 4 at $p = 5$, 5 at $p = 6$, 6 at $p = 7$, 9 at $p = 10$, 11 at $p = 12$, 19 at $p = 20$, 23 at $p = 24$), all eight self-reported the same figure the fabric logged, and all eight recorded $p - 1$ rounds. Both marker classes in Figure 1 lie on the closed-form line in both panels, and there are no red crosses at any size.

Token accounting is exact at all 21 sizes: the `msg.send` events sum to $p - 1$ tokens and the collective reports $p - 1$ as `tokens_sent`. A forwarding rank passes on `result`, the handle it received, with no wrapper, so the transport's and the algorithm's views of the volume cannot diverge. This is the property that fails in `reduce/chain`, which wraps its accumulator in a three-field envelope and charges only the accumulator, logging $16(p - 1)$ tokens while reporting $p - 1$.

Eight sizes were measured twice, under `mb-free` and `mb-smoke`. Every count agrees at all eight. The run wall times differ by factors from 1.01 to 1.75 — for instance 0.101 s against 0.057 s at $p = 12$ — which for this family is the largest disagreement anywhere in the set, and is exactly what one expects of a measurement whose dominant term is scheduler behaviour under contention.

6 When to choose this algorithm

Chain broadcast is defensible, which is a claim worth making carefully because the mirrored chain reduction is not. All three broadcast algorithms move $p - 1$ messages, so they differ only in depth, and the question is what depth costs. For a broadcast in AgentMPI it costs latency and nothing else, and this family is where the evidence is cleanest: across all thirteen sizes the set of distinct digests appearing in the collective's `msg.send` payloads has size exactly one, so at $p = 32$ thirty-one successive forwards deliver the digest rank 0 published, unmodified. `fold_depth` is 0 in all 32 `coll.bcast` records at that size, as it is at every other. A chain broadcast is not a game of telephone because forwarding moves a content-addressed handle and an interior rank has no opportunity to paraphrase. The mirrored measurement is the contrast: `reduce/chain` carries $p - 1$ *distinct* digests at every size — a new artifact for every one of its $p - 1$ messages — even though all p ranks contributed the identical value 1 under `SUM`, because each hop applies the operator and produces something new. Depth is recoverable for a broadcast and irrecoverable for a reduction, and that is why the same topology is a reasonable fallback here and a poor default there.

So the case for chain rests on the one thing it optimises: fan-out. Every rank sends at most one message and receives at most one, where `flat` concentrates $p - 1$ sends on the root and `binomial` gives the root $\lceil \log_2 p \rceil$ of them. That matters when forwarding costs the forwarding rank something the fabric does not see — specifically, when the harness admits the payload into the rank's context on the way through, in which case `flat`'s $p - 1$ simultaneous admissions and `binomial`'s staged fan-out are qualitatively different from chain's single admission per rank. This sweep cannot test that distinction: zero of 1,110 receptions across the six sweeps admitted a token.

The cost is severe and the sweep prices it exactly. Chain's critical path is $p-1$ hops against binomial's $\lceil \log_2 p \rceil$ steps, so the model predicts a ratio of 7/3 at $p = 8$, 15/4 at $p = 16$ and $31/5 = 6.2$ at $p = 32$. Measured maximum per-rank blocking gives 63.8 against 12.8 ms at $p = 8$ (a factor of 5.0), 177.2 against 25.0 ms at $p = 16$ (7.1), and 1,481.8 against 44.8 ms at $p = 32$ (33). The measured penalty exceeds the predicted one at every size and the gap widens, because the per-hop cost is itself growing with p. Against `flat` the comparison is worse still: 1,481.8 against 28.0 ms at $p = 32$, a factor of 53, because flat's leaves never wait for one another. My reading is that chain broadcast is the right choice only when p is small enough that $p - 1 \approx \lceil \log_2 p \rceil$ — which is to say $p \leq 4$, where the two differ by at most one hop — or when a per-rank fan-out of one is a hard constraint rather

than a preference. At every size in this sweep above $p = 5$, binomial dominates it on latency at no cost in traffic, in fidelity or in accounting.

7 Threats to this reading

These runs contain no agents, and for this family that cuts both ways. Every one of the 21 runs reports `executors = {"none": p}`, zero `agent.call` events, zero busy time, an idle fraction of 1.0 and $0.000 of cost. The 47.8 ms per hop at $p = 32$ is an SQLite transaction plus an event append plus a scheduler wake-up; the cost model's per-message α is on the order of 10^0–10^1 s, three orders of magnitude larger. So the measured chain-versus-tree ratio of 33 at $p = 32$ is not the ratio a real agent population would see. Under the model it would be 6.2, because α dominates and α is paid once per round by both. The measured ratio is larger than the modelled one only because the harness's per-hop cost grows with p while α would not, and a reader should quote the 6.2.

Against that, this is the one broadcast family where the wall times are not swamped by start-up. In `mb-free__coll-bcast-chain-32` the `rank.init` events span 1.1 to 68.6 ms while the run lasts 1,498.7 ms, so launch stagger is under 5% of the total; the same run under `flat` is 92.5 ms total against an 87 ms launch spread, where essentially none of the wall time is the collective. That contrast is why the chain family's durations can carry an argument and the flat family's cannot.

The superlinear per-hop cost is an inference and I have not isolated it. The claim is that the rise from 11.8 ms per hop at $p = 16$ to 47.8 at $p = 32$ comes from idle ranks polling the fabric, and the support is circumstantial: nothing in the algorithm changes with p, the same rise appears in `reduce/chain` (from 28.6 to 80.2 ms per hop over the same range), and the flat and binomial families — where far fewer ranks are blocked at once — do not show it. A run with the same p and a shorter chain, or a fabric instrumented for lock contention, would discriminate; neither exists in this archive, and I have not re-run anything.

Finally, several configurations are unexercised. All 21 runs used `root = 0`, so the wrap-around guard `if nxt != root` is only ever driven with the root at position zero, where it terminates the chain at rank $p - 1$; with a non-zero root the pipeline would wrap through rank 0 and the guard would fire mid-sequence, and no archived run tests that. All 21 used a one-token scalar payload, so the volume term $(p - 1)n$ is validated only at $n = 1$, and the pipelining advantage that makes chain broadcast worthwhile in MPI — overlapping the transfer of successive chunks of a large message — is not something this implementation does at all, so no size of payload would reveal it.